



Wrocław
University
of Science
and Technology



Asynchronous Messaging: SQS, SNS, and EventBridge

Cloud Programming (W04IST-SI0826G)

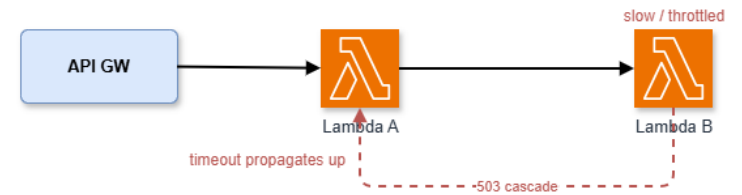
Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026



Why this matters



- One slow link stalls the entire chain
- One crash propagates upstream immediately
- Burst traffic hits the callee with no buffer



This week

After this class, you should be able to:

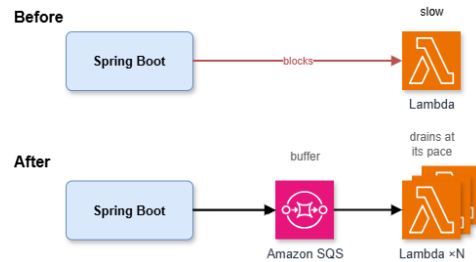
- **Explain** how asynchronous decoupling prevents cascade failures
- **Distinguish** SQS, SNS, and EventBridge and select the right tool
- **Configure** a Lambda event source mapping on an SQS queue
- **Design** a fan-out pipeline: SNS → SQS → Lambda
- **Apply** idempotency to a handler receiving at-least-once messages



Agenda

- 1 SQS: The Work Queue
- 2 SNS & EventBridge: The Broadcasters
- 3 Lambda as the Glue
- 4 Security

Break the chain: drop a queue



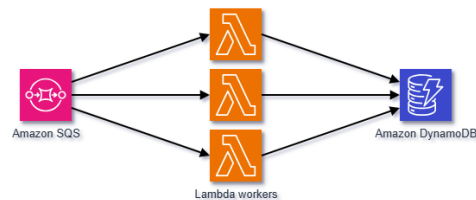
- Producer and consumer fully **decoupled** — neither knows the other
- Queue absorbs bursts; workers drain at sustainable throughput
- Consumer crash: messages wait; consumer restarts: messages resume

SQS queue model

- **Producer**: SendMessage — fire and forget
- **Consumer**: ReceiveMessage → process → DeleteMessage
- Queue persists independently of both endpoints
- Max message size: **256 KB** — max retention: **14 days**



Competing consumers — load leveling



- Multiple workers pull from the **same** queue — each gets a different message
- Lambda ESM scales out workers automatically with queue depth
- Burst fills the queue → workers scale up; quiet period → scale to zero
- Natural load leveler: **no coordination needed between workers**

Message lifecycle

1. SendMessage --> [in queue]
2. ReceiveMessage --> [invisible - visibility timeout starts]
- 3a. DeleteMessage --> removed permanently
- 3b. Timeout/crash --> back in queue --> retry

- **Visibility timeout**: window to process and delete — default **30 s**, max 12 h
- Set to **6x expected processing time** to avoid false re-deliveries
- No DeleteMessage → automatic retry — resilience without retry code



Dead Letter Queue

- After **maxReceiveCount** retries SQS routes message to a **DLQ**
- DLQ: a regular SQS queue of same type as the source queue
 - Standard → Standard, FIFO → FIFO
- Prevents a *poison message* from looping forever and starving valid work
- Workflow: inspect → fix the bug → redrive back to source queue

```
1 aws sqs set-queue-attributes --queue-url $QUEUE_URL \
2   --attributes '{
3     "RedrivePolicy": "{\"deadLetterTargetArn\":\"'$DLQ_ARN'\",
4     \"maxReceiveCount\": \"3\"}"
5   }'
```



Agenda

- 1 SQS: The Work Queue
- 2 SNS & EventBridge: The Broadcasters
- 3 Lambda as the Glue
- 4 Security



Standard vs FIFO — the trade-off

	Standard	FIFO
Throughput	Virtually unlimited	3 000 msg/s
Ordering	Best-effort	Strict per message group
Delivery	At-least-once	Exactly-once
Queue name	Any	Must end in .fifo

- **Message Group ID**: ordering partition — e.g. one group per customer account
- Default: **Standard** — prefer it unless strict ordering is a hard requirement
- FIFO throughput ceiling is real — Standard + idempotency beats FIFO for most workloads



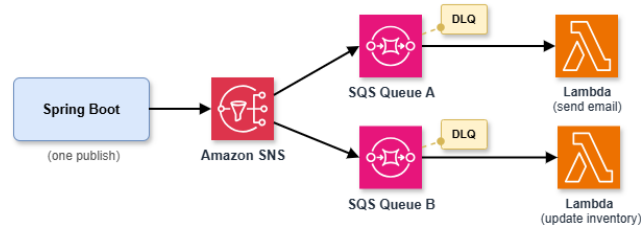
SNS: one publish, many consumers



- **Publisher** sends once to the topic — unaware of subscriber count or type
- Every subscriber gets an **independent copy** — failure in one does not affect others
- Supported protocols: SQS, Lambda, HTTP/HTTPS, email, SMS, mobile push



Fan-out: why SNS → SQS → Lambda



Why not directly from SNS to Lambda ?

- No buffering — burst hits Lambda immediately
- Retries opaque — managed inside Lambda async, not in an inspectable queue
- DLQ lives on Lambda destination, not the subscription

SQS adds: burst buffer + per-consumer DLQ + visible retry metrics



Subscription filter policies

- Default: **every** subscriber receives **every** message
 - **Filter policy:** JSON attribute match — subscriber receives only the matching subset
- ```
1 { "eventType": ["ORDER_PLACED"] }
```
- Defined on the **subscription**, not the topic — publisher code unchanged
  - Eliminates Lambda invocations for irrelevant message types



## Before we continue — predict

`ORDER_PLACED` is published to an SNS topic.  
Two SQS subscriptions exist: one for email, one for inventory.  
The inventory Lambda **crashes** while processing its copy.

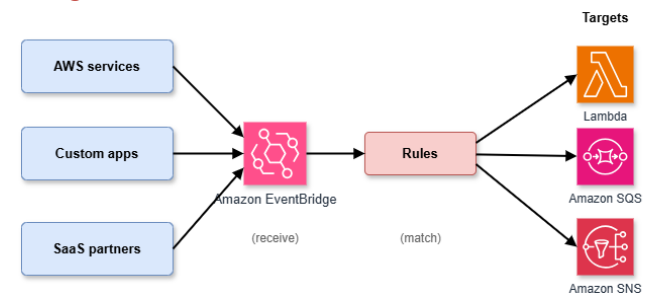
What happens to the email Lambda's copy?

- 1 Also retried — SNS resends to all subscribers on any failure
- 2 Unaffected — each subscriber queue holds an independent copy
- 3 Depends on the subscription filter policy

2 — Each subscription queue holds a separate copy. The inventory crash has zero effect on the email queue. This is the core value of fan-out: **independent failure domains**.



## EventBridge: the smart event bus



- **Event bus:** receives structured JSON events; rules route them to targets
- Default bus: all AWS service events arrive here automatically
- Custom bus: for your own application events
- Rules match on **any JSON field** — `source`, `detail-type`, `detail.*`



## EventBridge rules and scheduled targets

- Content-based routing rule:

```
1 {
2 "source": ["aws.s3"],
3 "detail-type": ["Object Created"],
4 "detail": { "bucket": { "name": ["my-uploads-bucket"] } }
5 }
```

- Scheduled invocation — direct upgrade from manual cron:

```
1 rate(5 minutes) --> Lambda (every 5 min)
2 cron(0 8 * * ? *) --> Lambda (08:00 UTC daily)
```

- Targets: Lambda, SQS, SNS, Step Functions, API destinations
- Scheduled rules invoke Lambda **asynchronously**



## SNS vs EventBridge

|                    | SNS                    | EventBridge           |
|--------------------|------------------------|-----------------------|
| Primary use        | Fan-out to queues/HTTP | Content-based routing |
| Routing logic      | One attribute filter   | Any JSON path         |
| Scheduled delivery | No                     | Yes (rate / cron)     |
| SaaS event sources | No                     | 200+ partners         |

- **Heuristic::**
  - SNS for fast broadcast to known subscribers
  - EventBridge when routing depends on payload content or a schedule
- In practice they layer: EventBridge routes → SNS topic → SQS queues → Lambda

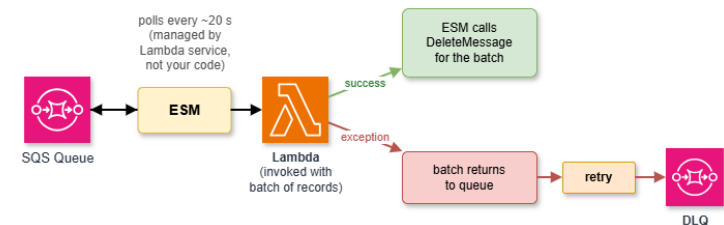


## Agenda

- 1 SQS: The Work Queue
- 2 SNS & EventBridge: The Broadcasters
- 3 Lambda as the Glue
- 4 Security



## SQS event source mapping



- **No polling code** in your function — ESM owns `ReceiveMessage` and `DeleteMessage`
- Batch delivered as `event["Records"]` — list of messages



## Batch tuning

| Parameter       | Default   | Guidance                                 |
|-----------------|-----------|------------------------------------------|
| Batch size      | 10        | Up to 10 000 — more = fewer invocations  |
| Batch window    | 0 s       | Up to 300 s — accumulate larger batches  |
| Max concurrency | Unlimited | Cap to protect downstream DB connections |

- Larger batch → fewer invocations → lower Lambda cost
- Larger batch window → higher latency — tune to your SLA
- **Max concurrency cap** =  $\text{workers} \times \text{connections-per-worker} \leq \text{DB max\_connections}$



## Partial batch failure

Default behaviour: one failed message fails the **entire batch** — all 10 retried.

Solution: `ReportBatchItemFailures`

```

1 def handler(event, context):
2 failures = []
3 for record in event["Records"]:
4 try:
5 process(record["body"])
6 except Exception:
7 failures.append({"itemIdentifier":
8 ↪ record["messageId"]})
9 return {"batchItemFailures": failures}

```

- Enable on ESM: `FunctionResponseTypes = [ReportBatchItemFailures]`
- Only the listed message IDs return to the queue — the rest are deleted



## Admin challenge

### Scenario

A Lambda processes SQS batches of 50 messages. One message in every batch is malformed JSON — `json.loads` raises an exception.

`ReportBatchItemFailures` is **not** enabled.

What is happening in production right now?

- All 50 messages retried on every batch → valid messages blocked
- After `maxReceiveCount` retries, all 50 move to the DLQ — 49 of them valid
- Zero throughput; DLQ fills with mostly-valid messages
- **Fix:** enable `ReportBatchItemFailures` + catch exceptions **per record**, not around the whole loop



## At-least-once delivery and idempotency I

- SQS Standard: **at-least-once** — rare but real duplicate deliveries
- Network retries mean your Lambda *will* receive the same message twice — eventually
- **Idempotent handler:** processing the same message twice produces the same outcome



## At-least-once delivery and idempotency II

Concrete fix — conditional write in DynamoDB:

```
1 table.put_item(
2 Item={"orderId": order_id, "status": "CONFIRMED"},
3 ConditionExpression="attribute_not_exists(orderId)"
4)
5 # Second call with same orderId --> ConditionCheckFailedException
 ↳ --> catch and ignore
```

- DynamoDB rejects the duplicate write — no double-charge, no second email
- FIFO eliminates SQS-level duplicates — but a Lambda crash after the effect still causes a retry

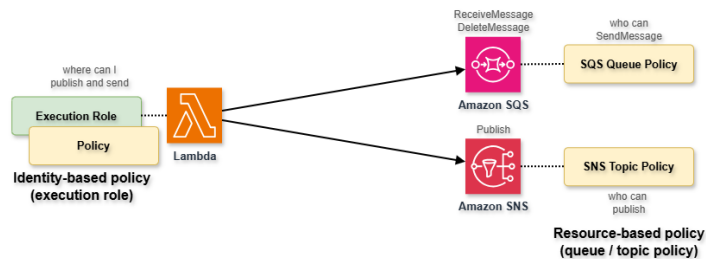


## Agenda

- 1 SQS: The Work Queue
- 2 SNS & EventBridge: The Broadcasters
- 3 Lambda as the Glue
- 4 Security



## IAM: two directions of trust



- **Execution role**: what Lambda is *allowed to do* outbound
- **Queue / topic policy**: who is *allowed to write* inbound
- Both must be correct — one missing → integration silently fails
- AWS Academy: attach permissions to **LabRole**; custom scoped roles require permission boundaries



## Encryption — quick reference

|               | SSE-SQS     | SSE-KMS                                       |
|---------------|-------------|-----------------------------------------------|
| Key ownership | AWS-managed | Customer CMK                                  |
| Cost          | Free        | \$1/month + API calls                         |
| AWS Academy   | Available   | Restricted — LabRole needs <code>kms:*</code> |

- In transit: HTTPS endpoint — enforced by default for all AWS SDK calls
- Lab recommendation: **SSE-SQS** — zero extra IAM setup required



## In the lab

You will:

- Create SQS queues and SNS topic; subscribe queues to the topic
- Configure Lambda ESM on each SQS queue
- Publish a test order from Spring Boot; trace end-to-end in CloudWatch Logs
- Inject a malformed message; observe batch failure; enable `ReportBatchItemFailures`



## References

- [1] Amazon Web Services, "Amazon SQS developer guide," 2026. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/> (accessed June 08, 2026).
- [2] Amazon Web Services, "Amazon SNS developer guide," 2026. <https://docs.aws.amazon.com/sns/latest/dg/> (accessed June 08, 2026).
- [3] Amazon Web Services, "Amazon EventBridge user guide," 2026. <https://docs.aws.amazon.com/eventbridge/latest/userguide/> (accessed June 08, 2026).